

Inside Out

Una aplicación web ~~por dentro y~~ por fuera



Quién soy



Juan María Hernández

- Desarrollador de software ~~viejo~~ con experiencia.
- Escribo un blog: <http://blog.koalite.com>
- Me quejo por todo en twitter: @gulnor
- Recopilo enlaces y camisetas: <http://kipple.koalite.com>

Qué vamos a hacer

Éste es un evento organizado por “*entusiastas por la tecnología de Microsoft*”, por tanto:

- No vamos a hablar de .NET.
- Ni de ninguna tecnología de Microsoft.
- **Vamos a conocer otra forma de trabajar.**

A photograph of two men with full beards and tattoos, dressed in hipster style. The man on the left wears a blue fedora, sunglasses, and a black t-shirt, holding a glass of beer. The man on the right wears a black fedora, sunglasses, and a black t-shirt. They are standing in what appears to be a bar or brewery. The text "¡Vamos a jugar a ser Hipsters!" is overlaid in the center.

**¡Vamos a jugar
a ser Hipsters!**

Dos partes



- 1) Pasar de IDEs para **construir una forma de trabajar optimizada** para nuestras necesidades.
- 2) Descartar el [infame] patrón MVVM para **diseñar UI más sencillos de entender**.



La vida sin un IDE

Node.js como IDE



- Node.js surge como una forma de ejecutar javascript en el servidor. A alguien le pareció una buena idea.
- A partir de eso, se empezaron a crear herramientas para mejorar el desarrollo y se aprovecharon para frontend.
- Nuestro nuevo IDE: **editor de texto + npm + CLI.**

Ejemplos

- Transpilers (babel, scss, less)
- Linters (jshint, eslint)
- Empaquetadores (browserify, webpack)
- Test runners (karma, mocha, nightwatch)
- Ofuscadores/minificadores (uglify)
- Gestores de tareas (grunt, gulp)



Vamos a usar

- Babel para usar ES2015 y JSX
- Browserify para modularizar la aplicación
- Watchify para recompilar automáticamente
- live-server como servidor de desarrollo



Paso a paso

{ Código }

Lo bueno

- Entorno más ligero
- Flexibilidad absoluta
- Multiplataforma (aunque “Windows Last”, claro)
- Un único lenguaje para todo



Lo malo

- Muchas (¿demasiadas?) opciones
- Configuración compleja
- Herramientas incompatibles entre ellas
- Infierno de versiones
(`npm shrinkwrap` es tu amigo)





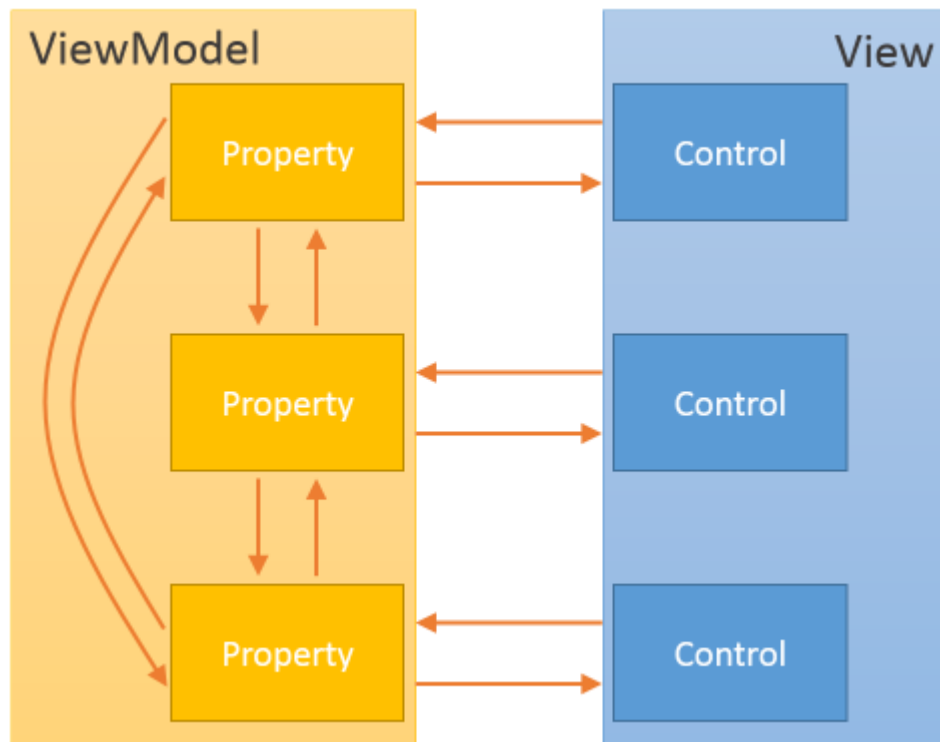
Adiós, MVVM



KILL IT! KILL IT WITH FIRE

La información en MVVM

- Mutabilidad y databinding bidireccional

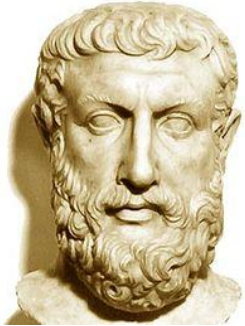


El problema de MVVM



- Toda la información es mutable.
- Todos los eventos pueden modificar toda la información.
- Reacciones en cadena.
- Dificultad para razonar sobre el estado del sistema.

Los griegos lo sabían

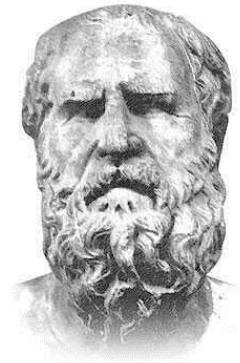


El ser es inmóvil y eterno

Parménides de Elea

La vida es devenir

Heráclito de Éfeso



Prog. Reactiva Funcional



Inmóvil y eterno:

$$f :: \text{state} \rightarrow \text{view}$$

Devenir:

$$g :: \text{state} \rightarrow \text{state}$$

* Esto es mentira, en realidad es un poco más complejo

El enfoque de ReactJS



- Es una librería, no un framework.
- Sólo cubre la “vista”. Ni routing, ni módulos, ni inyección de dependencias, ni nada más.
- Se basa en un flujo de información unidireccional. No se usa databinding bidireccional (aunque se puede).

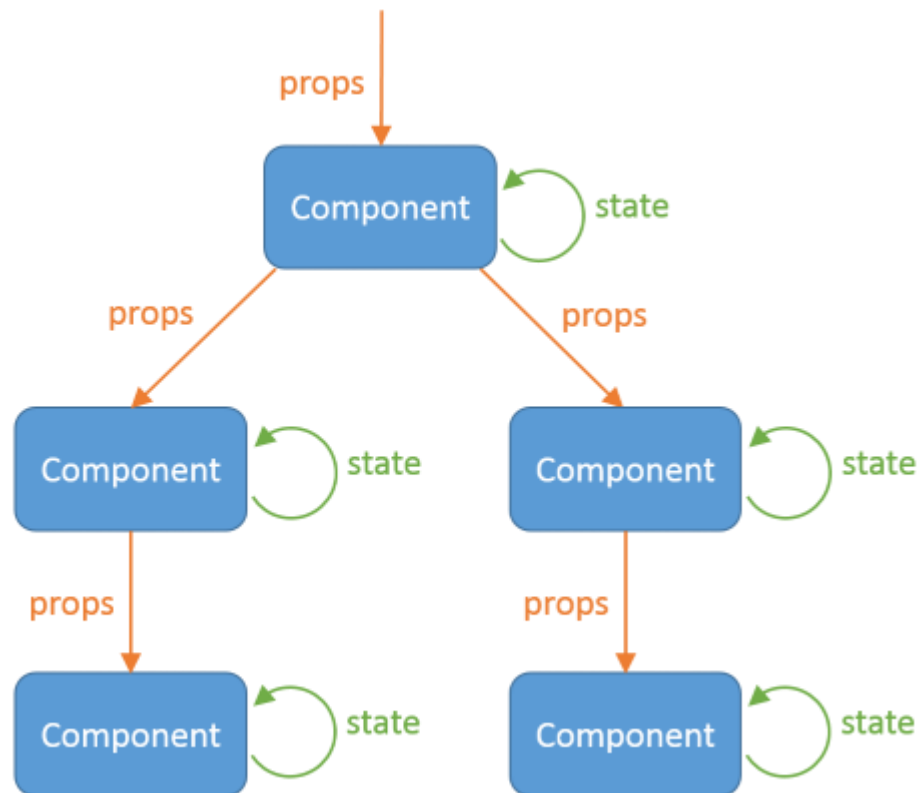
De qué se compone



- **Componentes:** vistas (o fragmentos de vistas) que convierten información en HTML.
- Dos tipos de información: **mutable e inmutable**. Con cada cambio del estado (mutable) se renderiza.
- **Virtual DOM** para optimizar el renderizado.
- Limitar al máximo el estado mutable a los componentes de más alto nivel.

Información en ReactJS

- Flujo de datos unidireccional



Virtual DOM



- Abstrae el DOM real.
- Se puede generar con objetos javascript o con JSX.
- JSX es una sintaxis específica para generar HTML.
 - Al principio a nadie le gusta.
 - Se acaba transpilando a JavaScript normal.

JSX → JavaScript

```
const name = 'Fulano';  
const greeter = <div>Hola {name}</div>;
```



```
const name = 'Fulano';  
const greeter = React.createElement(  
  'div', null, 'Hola', name);
```

Un ejemplo sencillo

```
{ Código }
```

How to draw an owl

1.



2.

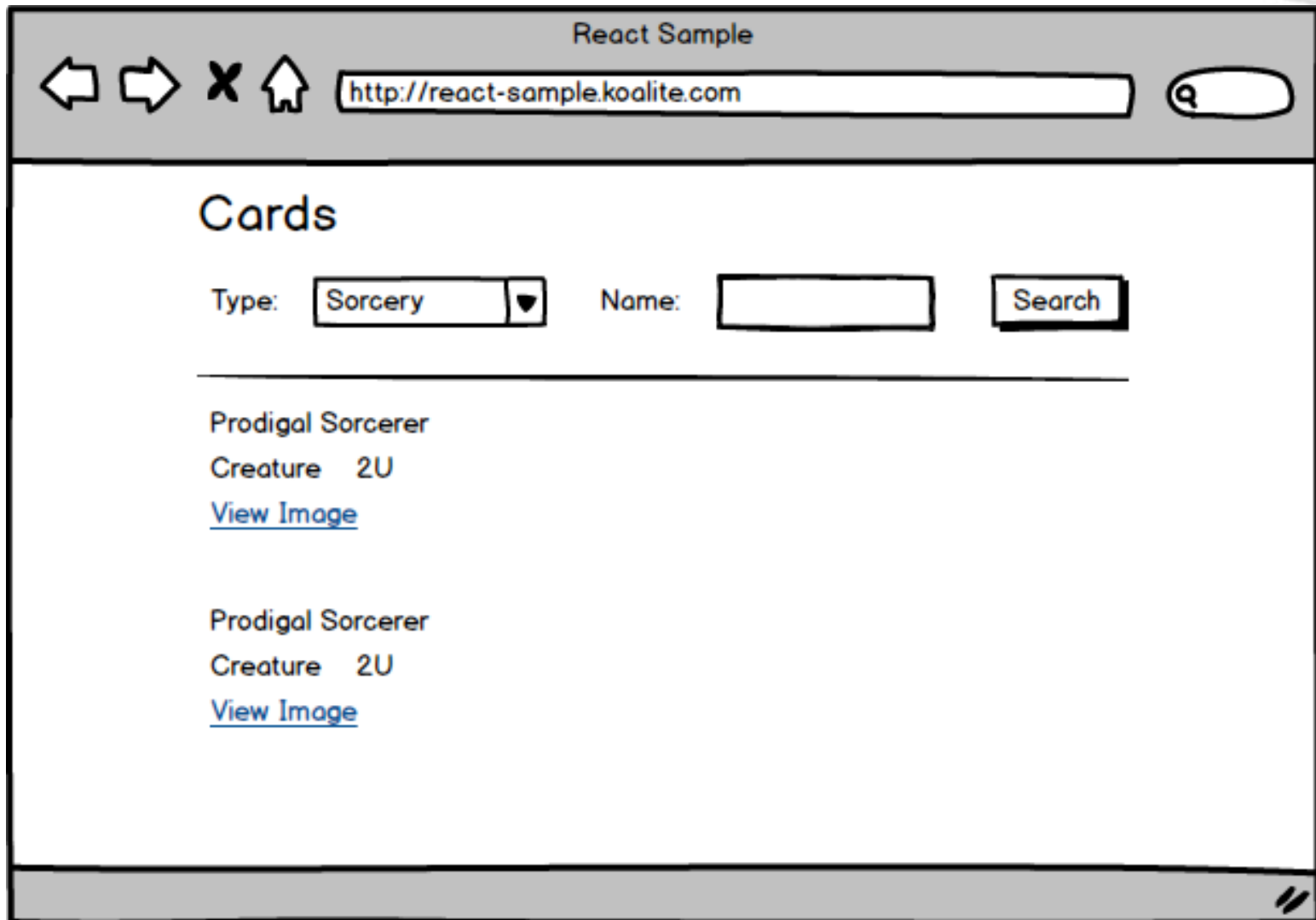


El resto es trivial

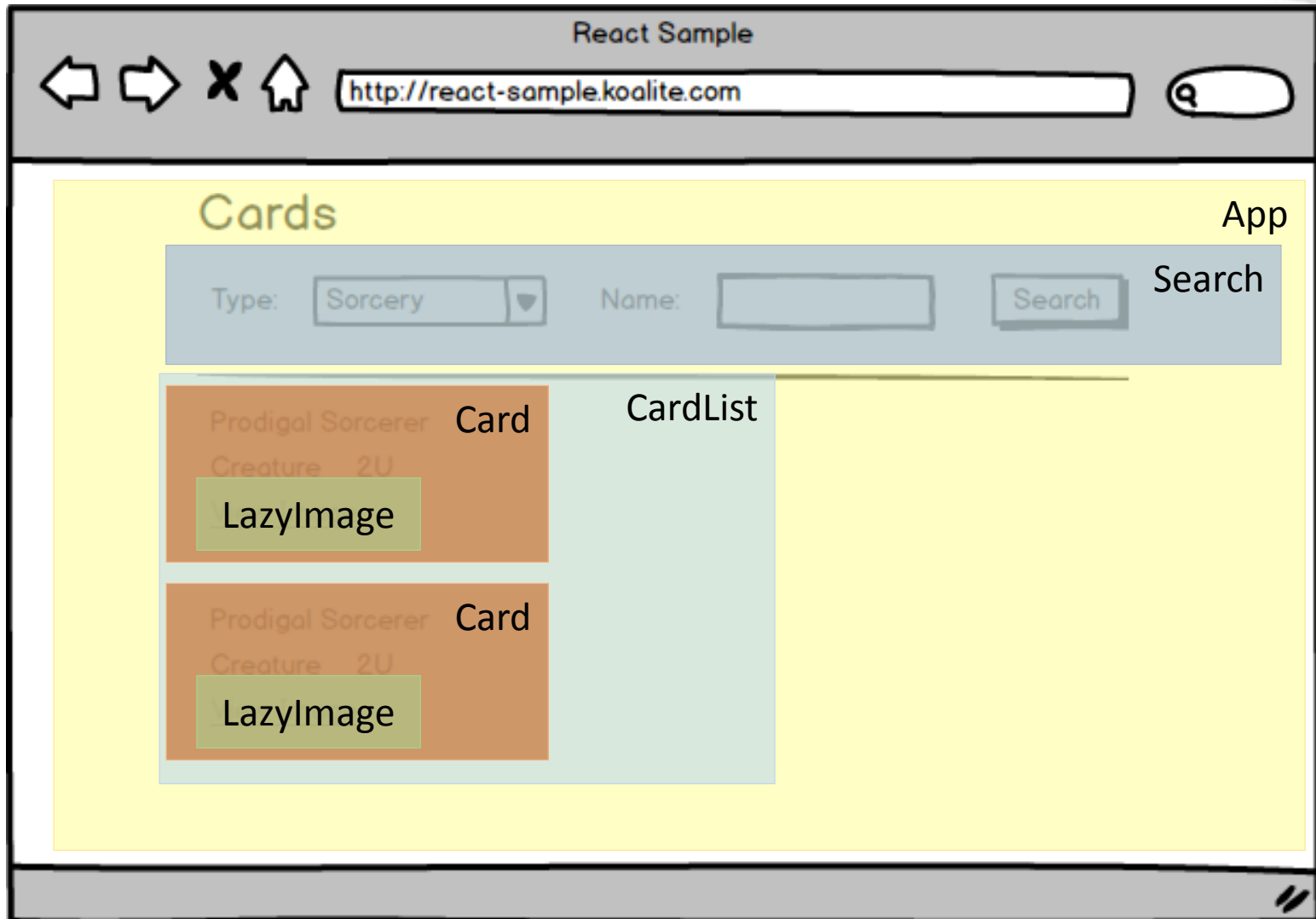
1. Draw some circles

2. Draw the rest of the fucking owl

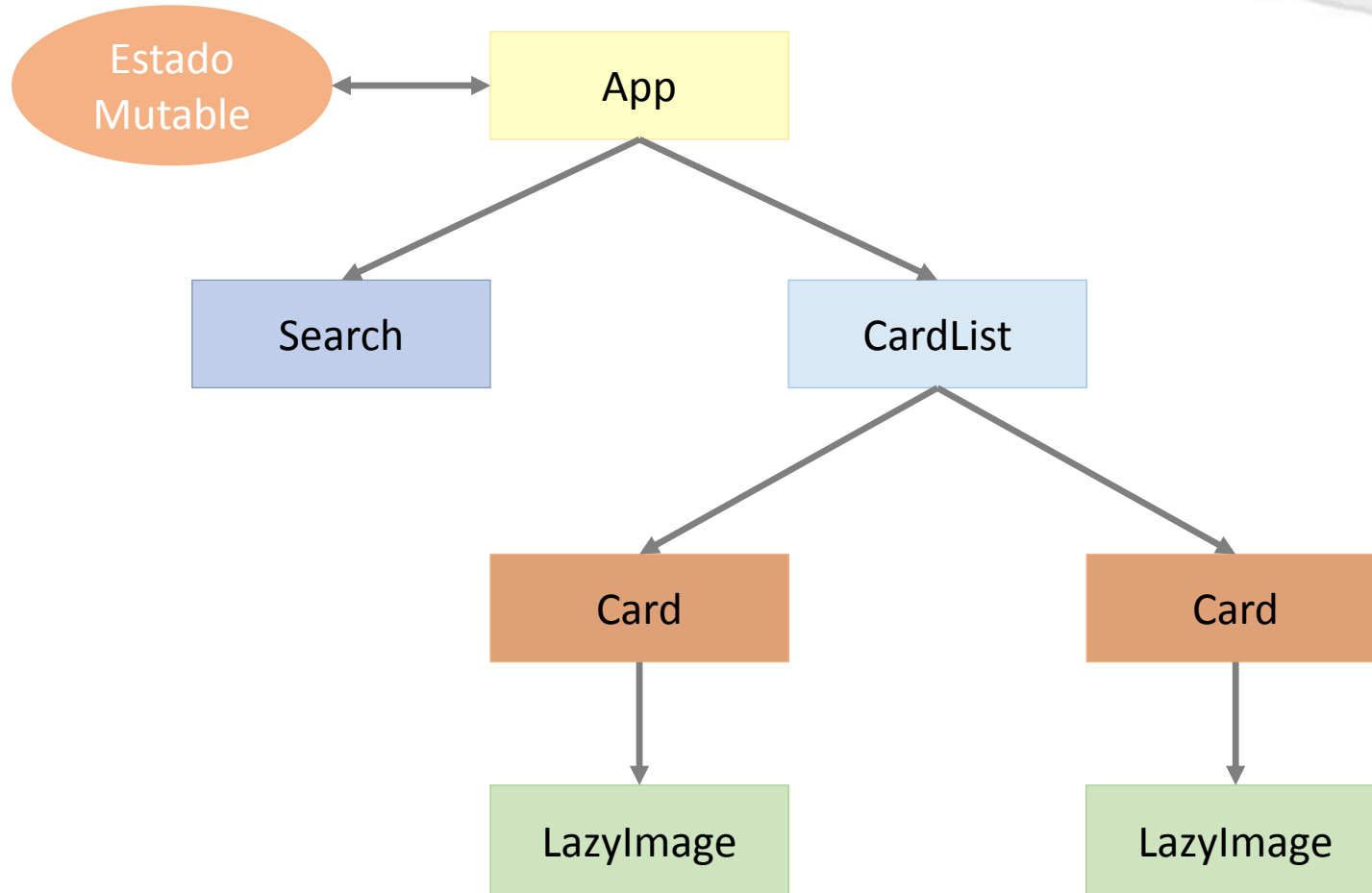
Algo más complejo



Componentes



Componentes



Código

{ Código }

<http://blog.koalite.com/tajamar/code.zip>

Qué nos dejamos

- Ciclo de vida de los componentes
- Mixins
- Routing con React-Router
- Flux
- GraphQL



Lo bueno



- Sólo es el UI, puedes usar lo que quieras para el resto.
- Facilita razonar sobre el flujo de información.
- Alta cohesión en cada componente. Similar a Polymer o WebComponents.
- Muy rápido gracias al Virtual DOM.

Lo malo



- Sólo es el UI, tienes que buscarte la vida para el resto.
- Obliga a pensar de una forma diferente que resulta extraña al principio.
- Requiere transpilar el JSX.
- Para aplicaciones de “forms over data”, el databinding bidireccional puede ser más cómodo.

Resumen

- Hay vida más allá de Visual Studio y las tecnologías apadrinadas por Microsoft. Ni mejor, ni peor: diferente.
- Se puede desarrollar con un editor de texto y herramientas de línea de comandos. Incluso hay gente (rara) que piensa que es mejor que un IDE.
- Podemos aplicar un enfoque funcional a la capa de presentación y evitar los problemas de MVVM.



Más información

node.js, npm y tooling variado

<http://blog.koalite.com/tag/grunt/>

<http://blog.koalite.com/2015/06/grunt-o-gulp-que-uso/>

<http://blog.koalite.com/2015/12/ni-grunt-ni-gulp-solo-npm/>

ReactJS

<http://blog.koalite.com/tag/reactjs/>



¡Gracias!